

Praxisaufgabe: Entwicklung von Agent Skills für die automatisierte UI-Konvertierung

Zielsetzung:

In dieser Aufgabe schlüpft ihr in die Rolle von "AI Engineers" und "Systemarchitekten". Euer Ziel ist es, einen standardisierten **Agent Skill** zu entwickeln, der einen rohen HTML/CSS-Prototyp für die spätere automatisierte Konvertierung in eine React Native- oder Flutter-App vorbereitet. Der Skill soll den Prototyp deduplizieren, die UI-Elemente nach dem Atomic-Design-Prinzip klassifizieren, wiederkehrende Bausteine konsolidieren und das "Gedächtnis" des Agenten in Form einer `workflow-progress.json` aufbauen.

Hintergrund:

Große Language Models (LLMs) können nicht tausende Zeilen HTML-Code auf einmal verarbeiten (Token-Limits). Wir nutzen daher das Prinzip der **Progressive Disclosure** (schrittweise Informationsfreigabe). Der KI-Agent (Manager Agent) ruft für speicherintensive Aufgaben lokale Skripte auf und verlässt sich auf eine strukturierte JSON-Datei als Checkpoint-System (`workflow-progress.json`).

✅ Voraussetzungen

Bevor ihr mit den Teilaufgaben beginnt, muss ein **HTML-Prototyp** für eure App vorhanden sein. Der Prototyp muss mindestens folgende Seiten (Screens) umfassen:

Screen	Beschreibung
Landing Page / Getting Started	Einstiegsseite der App (Willkommensscreen, Onboarding)
Registration	Registrierungsformular für neue Benutzer
Login	Anmeldeformular für bestehende Benutzer
Overview	Hauptübersicht der App (analog zum mitgelieferten <code>screen.png</code>)

Der Prototyp ist im Projekt unter folgendem Pfad abzulegen und entsprechend zu strukturieren:

```
docs/ui/prototypes/[app-name]/
├── auth/
│   ├── landing.html      # Landing Page / Getting Started
│   ├── registration.html # Registrierung
│   └── login.html        # Login
└── overview/
    └── overview.html     # Hauptübersicht (Referenz: screen.png)
```



Hinweis: Ersetzt `[app-name]` durch den Namen eurer App (z. B. `workout-tracker`). Die HTML-Dateien müssen vollständiges HTML/CSS enthalten und im Browser darstellbar sein. Externe CSS-Frameworks (z. B. Bootstrap, Tailwind) sind erlaubt, sofern sie lokal eingebunden sind.

Die Teilaufgaben

Aufgabe 1: Pre-Processing, Deduplizierung & ID-Injektion (Das Skript)

Der Agent benötigt ein Skript, das die "Schwerstarbeit" am Dateisystem übernimmt, um das Kontext-Fenster der KI zu schonen.

Erstellt ein Skript (z. B. in Python oder JavaScript), das im Unterordner `scripts/` eures Skills liegt. Das Skript muss folgende Anforderungen erfüllen:

1. **ID-Injektion (Traceability):** Parses das HTML und fügt jedem relevanten (sichtbaren) HTML-Knoten ein eindeutiges Data-Attribut hinzu (z. B. `cdd-id="atm-btn-01"` oder `cdd-id="mol-card-01"`).
2. **Deduplizierung:** Scannt alle HTML-Dateien des Prototyp-Ordners und filtert exakte Duplikate heraus (z.B. mittels Content-Hashing), weitere Duplikate sollen durch Referenzen über die injizierte `cdd-id` referenziert werden
3. **JSON-Output:** Das Skript soll die gefundenen und markierten Elemente nicht selbst beurteilen, sondern als JSON-Struktur für den Agenten bereitstellen.

Aufgabe 2: CDD Klassifizierungs-Logik (Die SKILL.md)

Erstellt die zentrale Instruktionsdatei (`SKILL.md`) für den Manager Agent. Diese Datei ist das "Gehirn" des Skills und muss in YAML-Frontmatter und Markdown-Instruktionen unterteilt sein.

Definiert darin für den Agenten glasklare, deterministische **Regeln zur Klassifizierung**:

1. **Atome:** Definiert, woran der Agent ein Atom erkennt
2. **Moleküle:** Definiert die Erkennung von Molekülen
3. **Organismen:** Definiert die Regeln für Organismen

Definiere den Skill so, dass er die zuvor definierten Scripte und den durch die Scripte erzeugten json-Output verwendet um anhand der definierten Regeln die Klassifizierung der identifizierten Komponenten vorzunehmen.

Ziel ist es, dass mit Hilfe des Skills ein output generiert wird, der es leichtgewichtigen bzw. günstigen LLM's ermöglicht mit Hilfe dieser Instruktionen/Daten die Implementierung der Komponenten für das angegebene Zielframework (in unserem Fall react-Native) durchzuführen.

Aufgabe 3: Konsolidierung & Workflow Checkpointing

Im HTML-Prototyp tauchen gleiche Buttons oder Karten (Cards) oft dutzendfach auf. Der Agent darf diese nicht als Dutzende eigenständige Komponenten konvertieren, sondern soll das zuvor erstellte json-Script verwenden.

Erweitert die `SKILL.md` um Instruktionen zur Konsolidierung und zum Aufbau der `workflow-progress.json`:

- **Current State:** aktuelle Stand der Konvertierung
- **Delta-Updates:** Beschreibt, wie der Agent die `workflow-progress.json` aktualisieren soll. Was passiert, wenn das Parsing-Skript aus Aufgabe 1 beim nächsten Durchlauf geänderte Datei-Hashes meldet?

Erwartete Abgabe (Deliverables)

Eure Abgabe muss der strikten Ordnerstruktur des offiziellen **Agent Skills Standards** entsprechen:

```
cdd-analysis-skill/
├── SKILL.md                # Enthält YAML-Metadaten und Prompt-
Instruktionen (Aufgabe 2 & 3)
├── scripts/
│   └── deduplicate_and_parse.py (o.ä.) # Das Skript für ID-Injektion und
Deduplizierung (Aufgabe 1)
└── references/
    ├── workflow-progress.template.json # Eine Muster-JSON, die das
    Endergebnis der Konsolidierung zeigt
    └── deduplicated-components.template.json # Datei, welche die depluzierten und
    kassifizierten Komponenten speichert
```

Zusätzlich sind folgende Artefakte als Teil der Abgabe verpflichtend:

- die im Arbeitsprozess erzeugte **workflow-progress.json** bzw. **work-progress.json** als tatsächlicher Arbeitsstand des Agenten
- eine ZIP-Datei mit den indizierten HTML-Prototypen, in denen die injizierten **data-agent-id**-Attribute bereits enthalten sind

Die ZIP-Datei muss den verarbeiteten Prototyp-Stand enthalten, also genau jene HTML-Dateien, die durch euer Pre-Processing-Skript analysiert, markiert und für die spätere Konvertierung vorbereitet wurden.

Zusatz-Herausforderung (Bonus):

Erklärt in einem kurzen Kommentarblock in der **workflow-progress.json**, wie das Tracking-System aus **dependencies** (z. B. Atom ändert sich -> Molekül muss aktualisiert werden) dem Agenten dabei hilft, **nur** die Komponenten neu zu generieren, die sich bei einem Update des HTML-Prototyps tatsächlich verändert haben.

Tipps für die Umsetzung

- **Progressive Disclosure:** Haltet die **SKILL.md** präzise (unter 500 Zeilen). Lagert große Konfigurationsvorgaben in den Ordner **/references** aus.
- **Keine Halluzinationen:** Je klarer eure Regeln für das Erkennen von Atomen und Molekülen sind, desto weniger "rät" die KI. Zwingt die KI in euren Prompts dazu, sich strikt auf die injizierten **cdd-ids** zu beziehen.
- **Web to Native Mapping:** Denkt daran, dass React Native keine **<div>**-Tags kennt. Eure Konsolidierung bereitet den perfekten Boden, damit der nächste Agent im Workflow aus euren Atomen saubere **<View>**- und **<Text>**-Komponenten bauen kann.